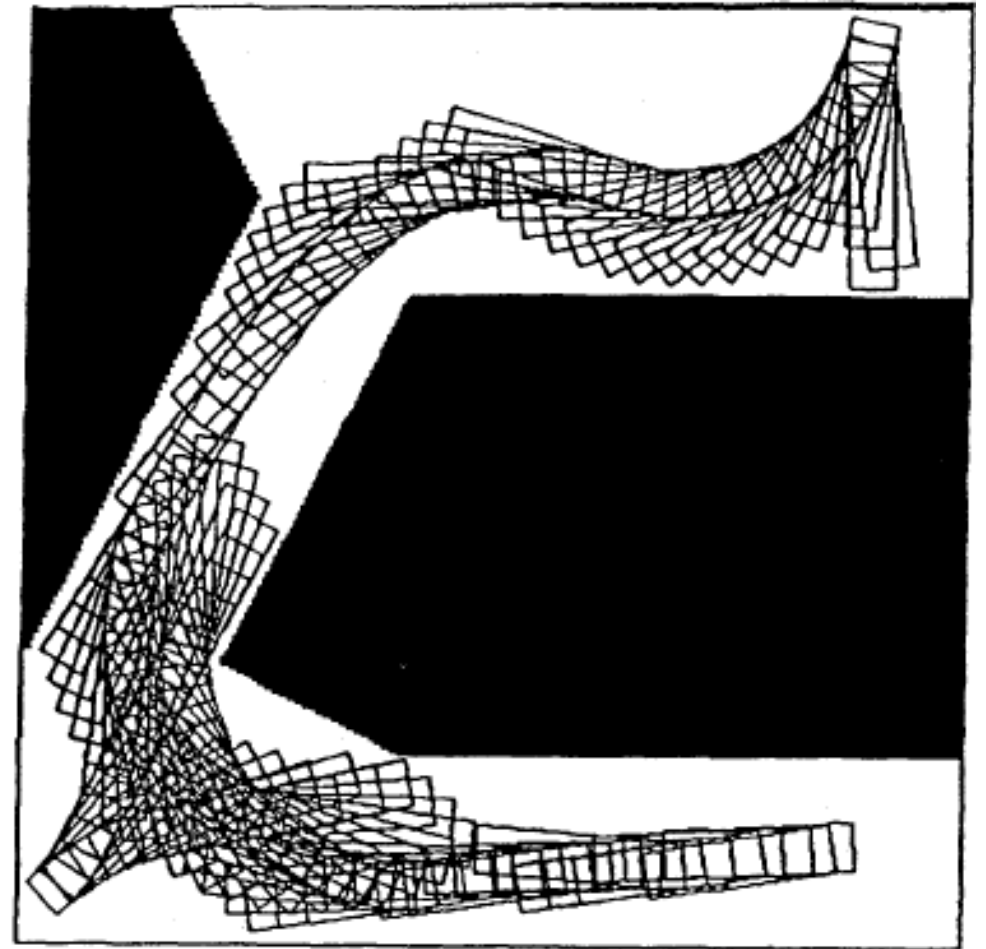
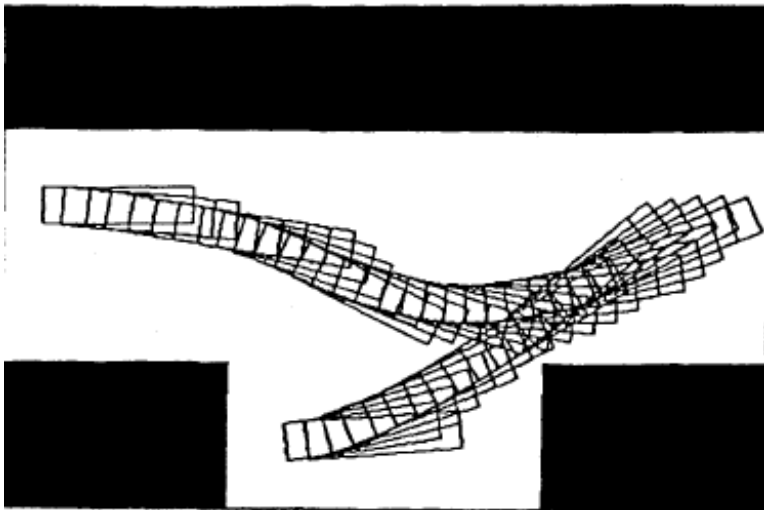


Motion Planning with Differential Constraints

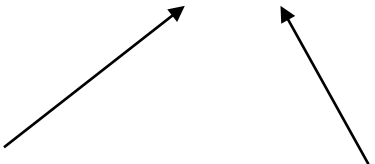
(Kinodynamic Planning)



Paths for a Car-Like Robot

Differential constraints

Dynamics, nonholonomic systems

$$\dot{x} = f(x, u)$$


The diagram shows two arrows pointing upwards towards the arguments of the function f in the equation $\dot{x} = f(x, u)$. The left arrow originates from the text "state vector" and points to the x argument. The right arrow originates from the text "controls" and points to the u argument.

state vector controls

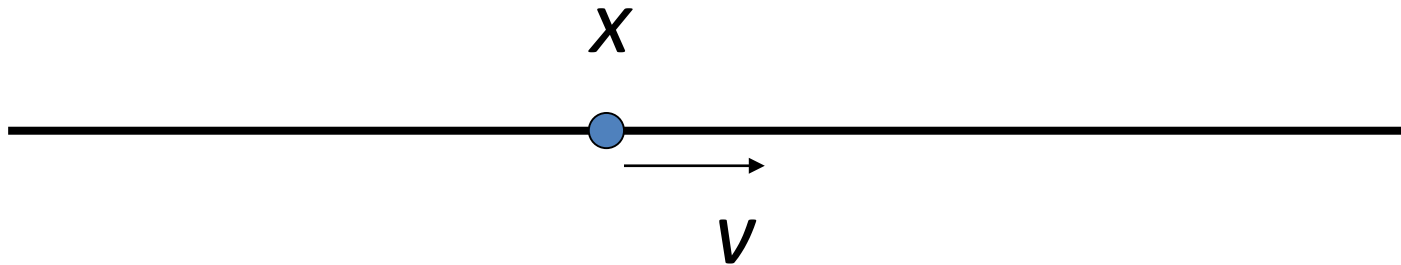
We'll consider fewer control dimensions than state dimensions

Euler Integration yields

$$f(x, u) = \dot{x} = \frac{dx}{dt} \approx \frac{x(\Delta t) - x(0)}{\Delta t}$$

$$x(\Delta t) \approx x(0) + \Delta t f(x(0), u(0))$$

Example: 1D Point Mass



Mass M

2D state space

Controlled force f

Equations of motion:

$$\dot{x} = \begin{pmatrix} \dot{x}_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} x_2 \\ f/M \end{pmatrix} = f(x, u)$$

$$dx/dt = v$$

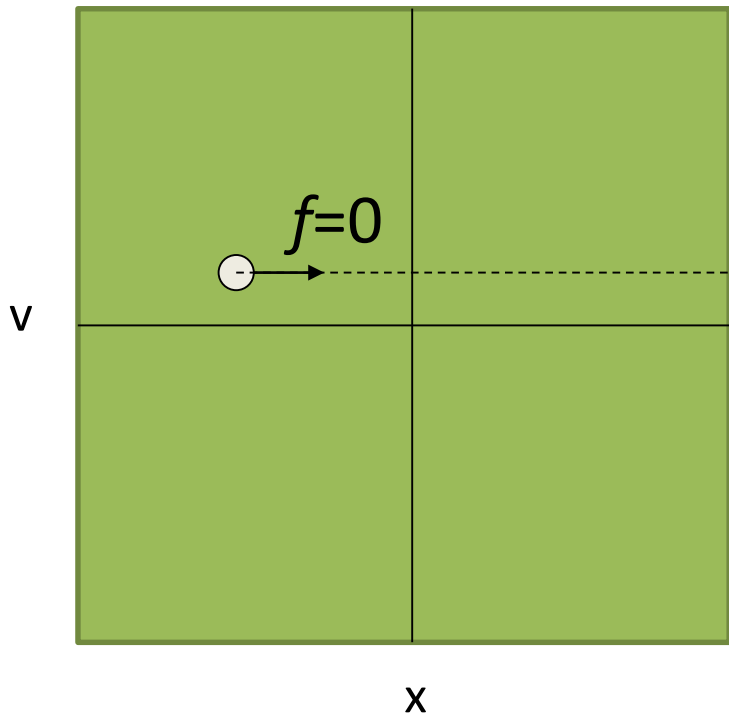
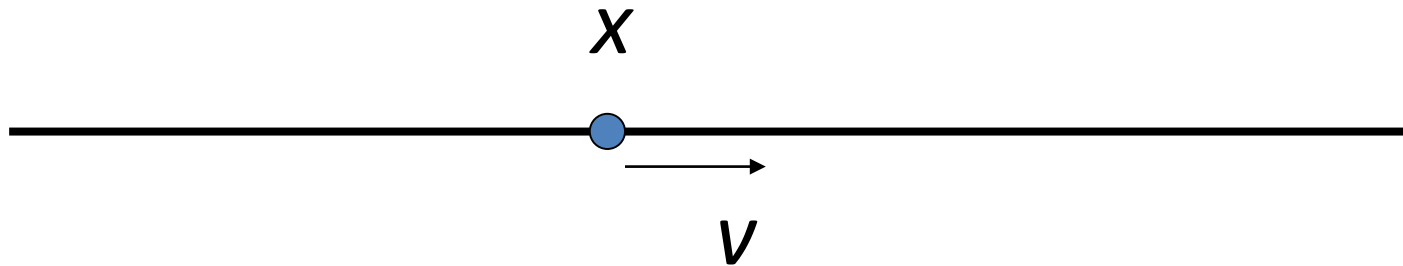
$$dv/dt = f / M$$

With the correspondence

$u=f$

$x_1 = x$ and $x_2 = v$

Example: 1D Point Mass



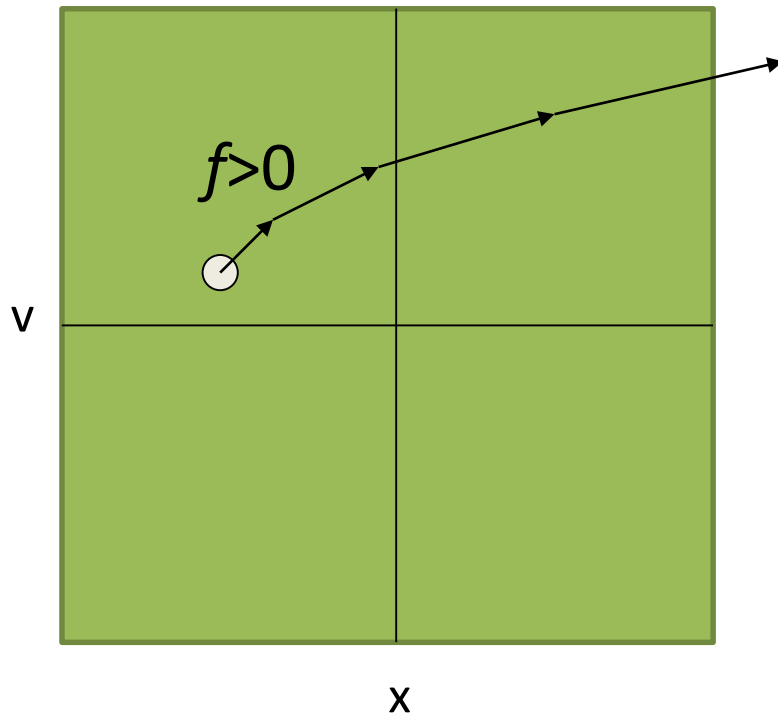
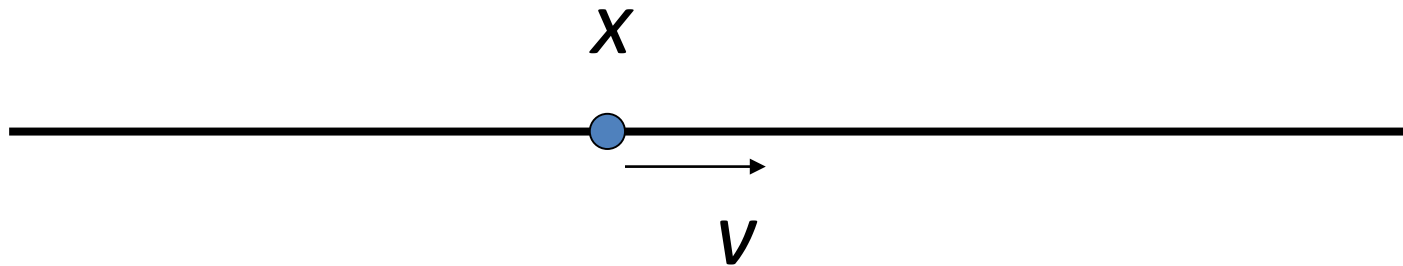
$$\begin{aligned} dx/dt &= v \\ dv/dt &= f / M \end{aligned}$$

Solution:

$$v(t) = v(0) + (f/M) * dt$$

$$x(t) = x(0) + v(0) * dt + \frac{1}{2} (f/M) * dt^2$$

Example: 1D Point Mass



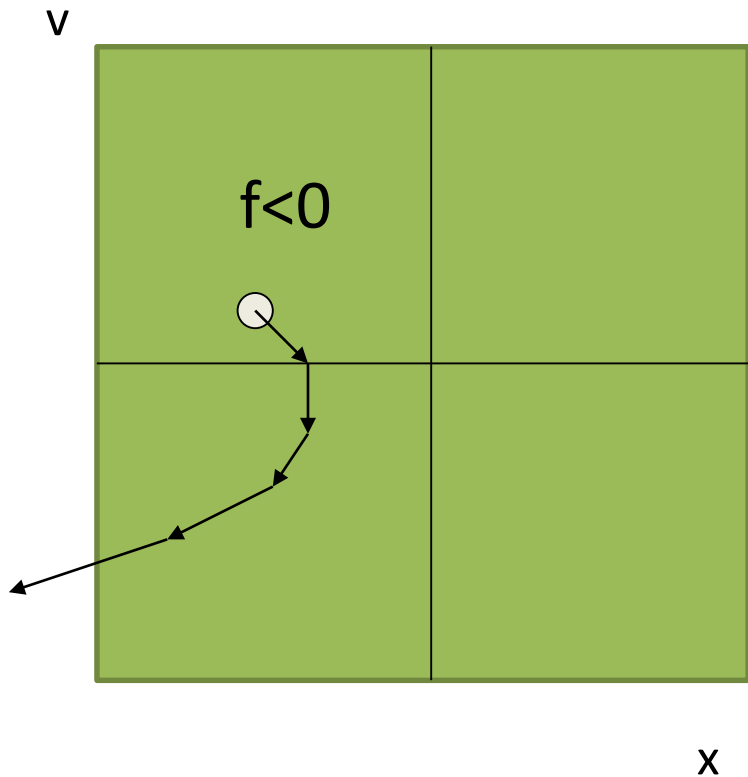
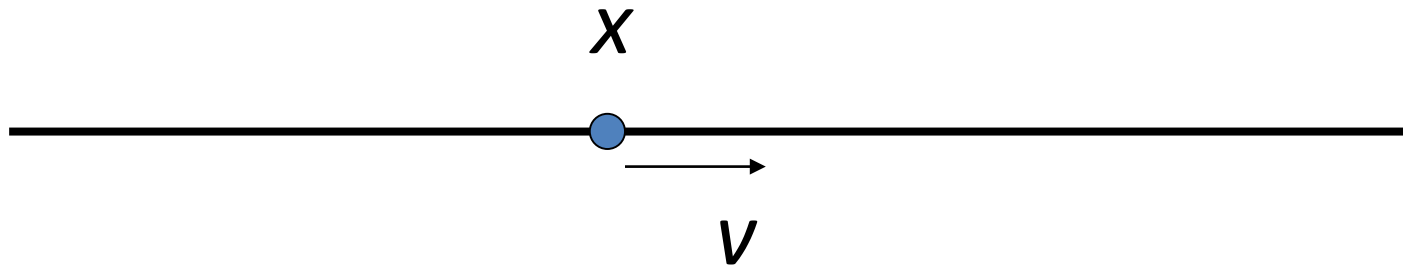
$$\begin{aligned} dx/dt &= v \\ dv/dt &= f / M \end{aligned}$$

Solution:

$$v(t) = v(0) + (f/M) * dt$$

$$x(t) = x(0) + v(0) * dt + \frac{1}{2} (f/M) * dt^2$$

Example: 1D Point Mass



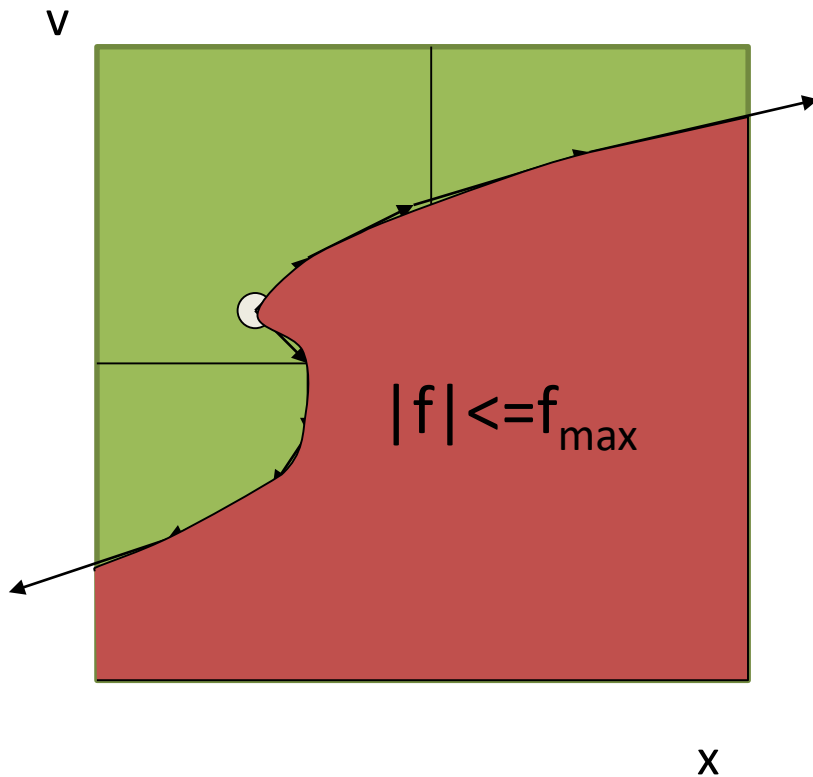
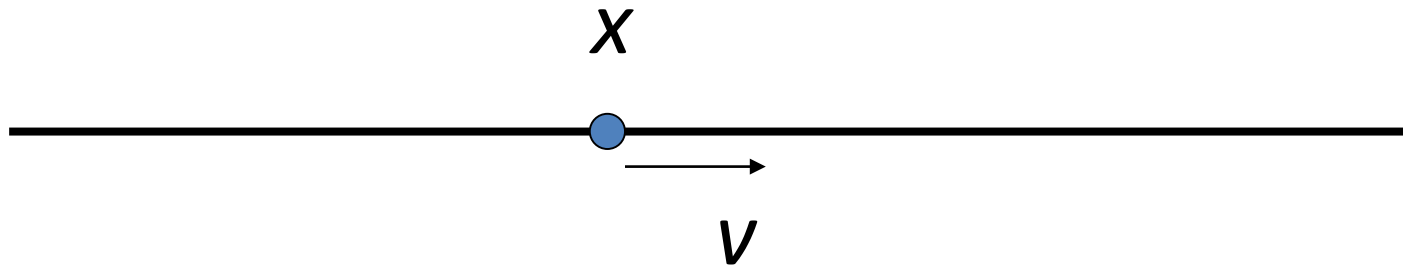
$$\begin{aligned} dx/dt &= v \\ dv/dt &= f / M \end{aligned}$$

Solution:

$$v(t) = v(0) + (f/M) * t$$

$$x(t) = x(0) + v(0) * dt + \frac{1}{2} (f/M) * dt^2$$

Example: 1D Point Mass



$$\begin{aligned} dx/dt &= dv \\ dv/dt &= f / M \end{aligned}$$

Solution:

$$v(t) = v(0) + (f/M) * dt$$

$$x(t) = x(0) + v(0) * dt + \frac{1}{2} (f/M) * dt^2$$

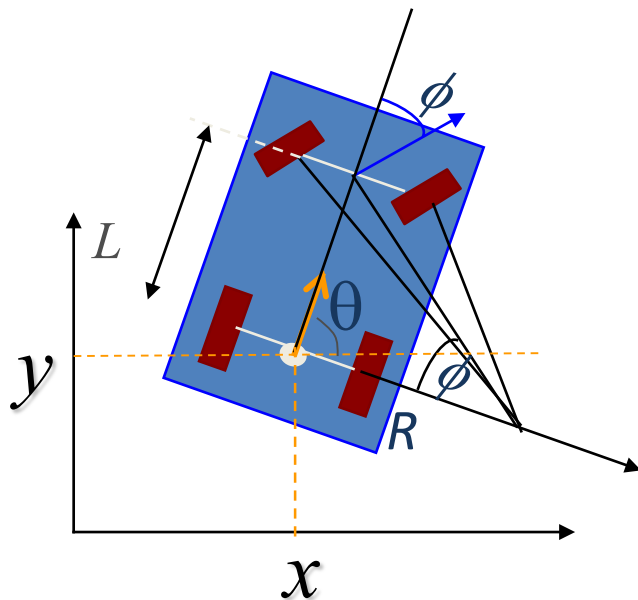
Under-Actuated Robots

- Fewer controls than dimensions in configuration space
- What is a degree of freedom: number of dimensions of C-space (global) or number of controls (local)?
- How can m controls make it possible to span a C-space with $n > m$ dimensions?
- By exploiting mechanics properties:
- Rolling-with-no-sliding contact (friction), e.g.: car, bicycle, roller skate
- Others: submarine, plane, object pushing

Why is it useful?

- Fewer actuators (less weight)
- Design simplicity
- Convenience (think about driving a car with 3 controls!)

Example: Car-Like Robot



$$\left. \begin{aligned} dx/dt &= v \cos \theta \\ dy/dt &= v \sin \theta \end{aligned} \right\} \rightarrow dx \sin \theta - dy \cos \theta = 0$$

$$d\theta/dt = v/R = (v/L)(L/R) = (v/L) \tan \phi$$

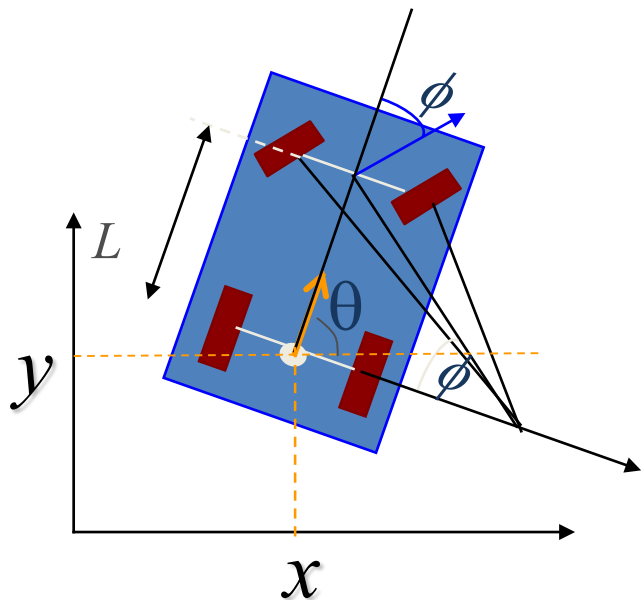
$$|\phi| \leq \Phi$$

$$v = ds/dt = R * d\theta/dt$$

Configuration space is 3-dimensional: $q = (x, y, \theta)$

But control space is 2-dimensional: (v, ϕ) with $|v| = \sqrt{(dx/dt)^2 + (dy/dt)^2}$

Example: Car-Like Robot



$$dx/dt = v \cos \theta$$

$$dy/dt = v \sin \theta$$

$$d\theta/dt = (v/L) \tan \phi$$

$$|\phi| \leq \Phi$$

$$dx \sin \theta - dy \cos \theta = 0$$

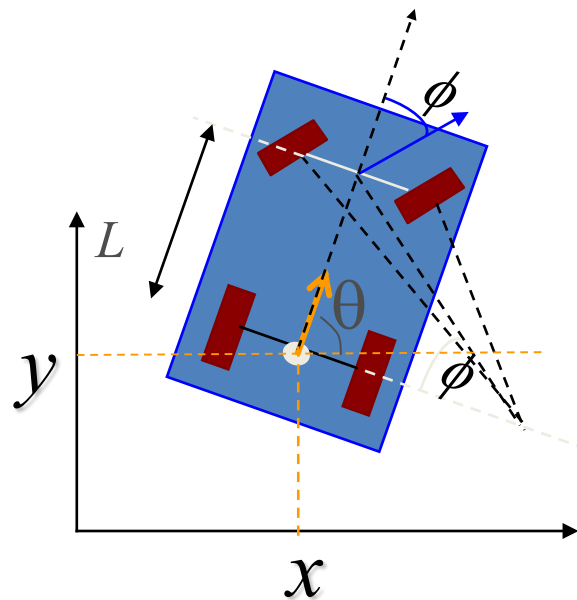
$$q = (x, y, \theta)$$

$$q' = dq/dt = (dx/dt, dy/dt, d\theta/dt)$$

$$dx \sin \theta - dy \cos \theta = 0 \quad \text{is a particular form of} \quad f(q, q') = 0$$

A robot is **nonholonomic** if its motion is constrained by a non-integrable equation of the form $f(q, q') = 0$

Example: Car-Like Robot



$$\left. \begin{aligned} dx/dt &= v \cos \theta \\ dy/dt &= v \sin \theta \end{aligned} \right\} \rightarrow dx \sin \theta - dy \cos \theta = 0$$

$$d\theta/dt = (v/L) \tan \phi$$

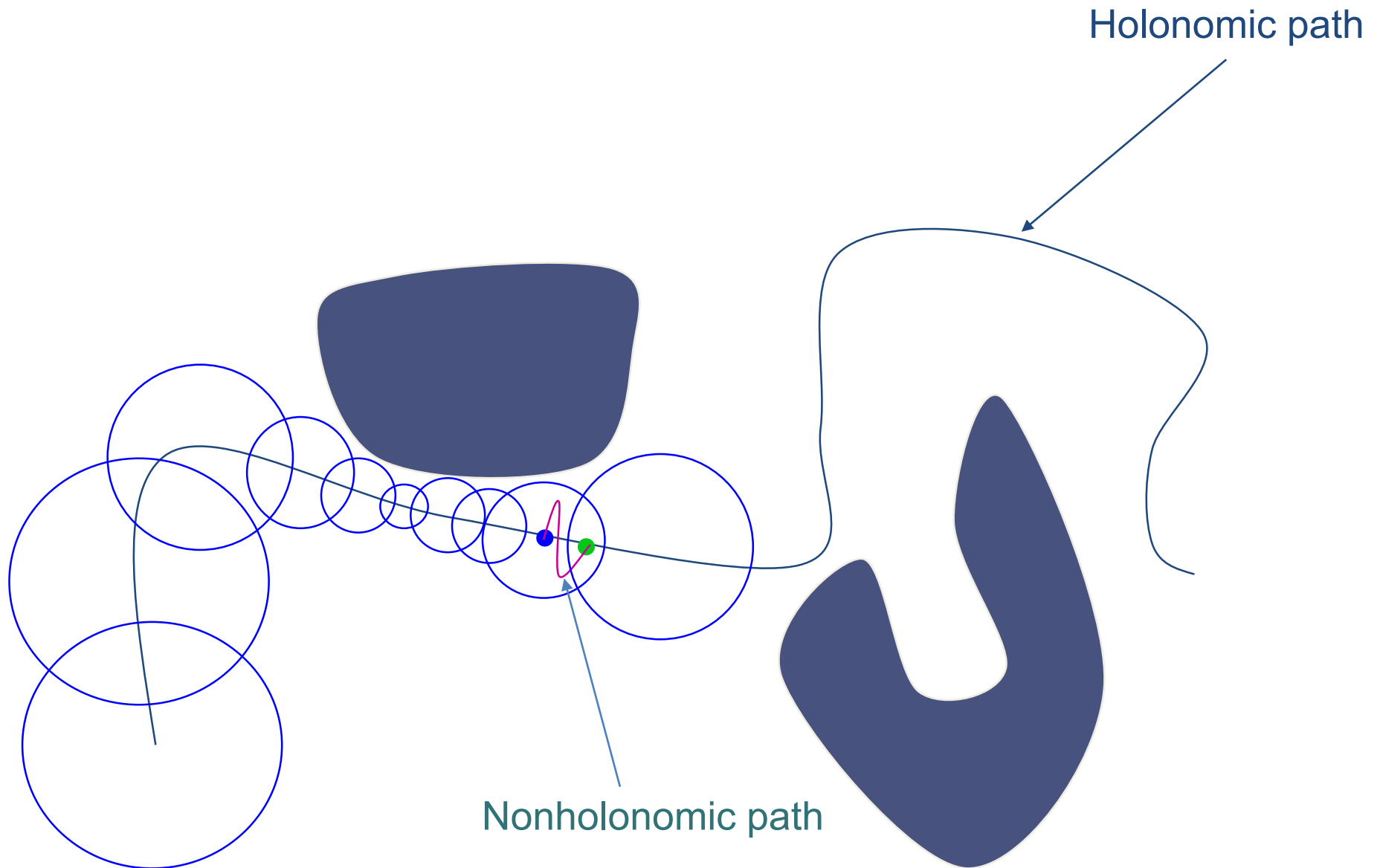
$$|\phi| \leq \Phi$$

Lower-bounded turning radius

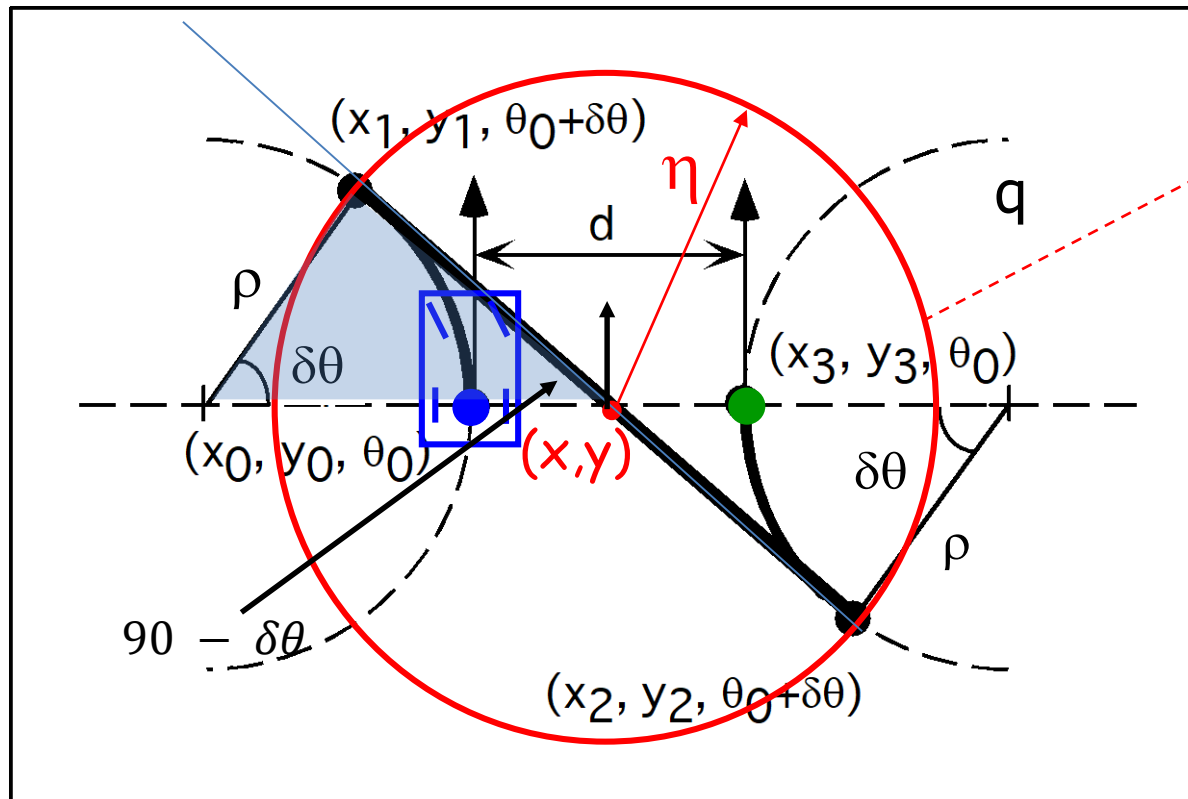
Nonholonomic Path Planning Approaches

- Two-phase planning (path deformation):
 - Compute collision-free path ignoring nonholonomic constraints
 - Transform this path into a nonholonomic one
 - Efficient, but possible only if robot is “controllable”
 - Need for a “good” set of maneuvers
- Direct planning (control-based sampling):
 - Use “control-based” sampling to generate a tree of milestones until one is close enough to the goal (deterministic or randomized)
 - Robot need not be controllable
 - Applicable to high-dimensional C-spaces

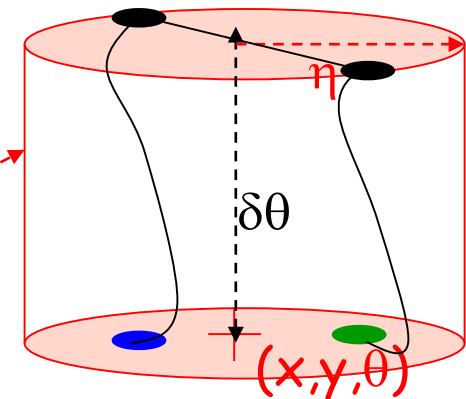
Path Deformation



Type 1 Maneuver



$CYL(x, y, \delta\theta, \eta)$



$$\eta = \rho \tan \delta\theta$$

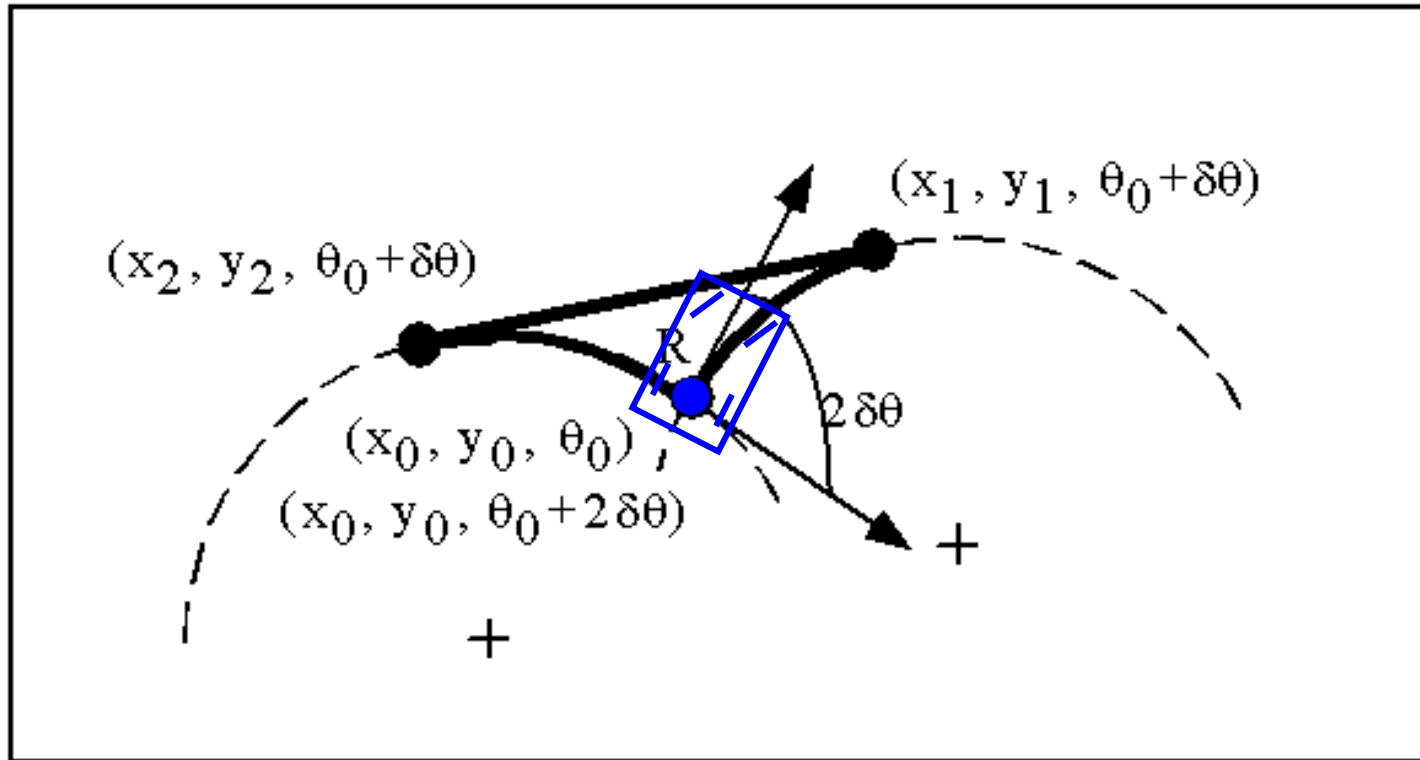
$$d = 2\rho(1/\cos \delta\theta - 1) > 0$$

$$\text{from: } \cos \delta\theta = \frac{\rho}{0.5d + \rho}$$

→ Allows sidewise motion

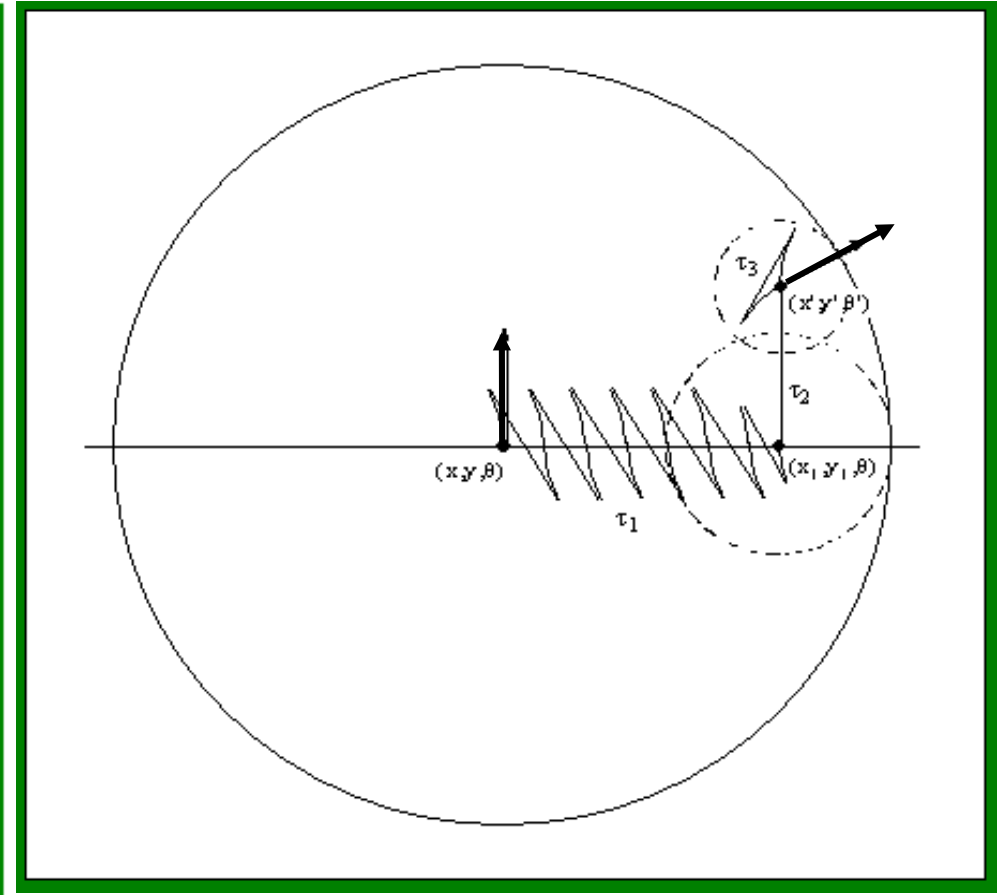
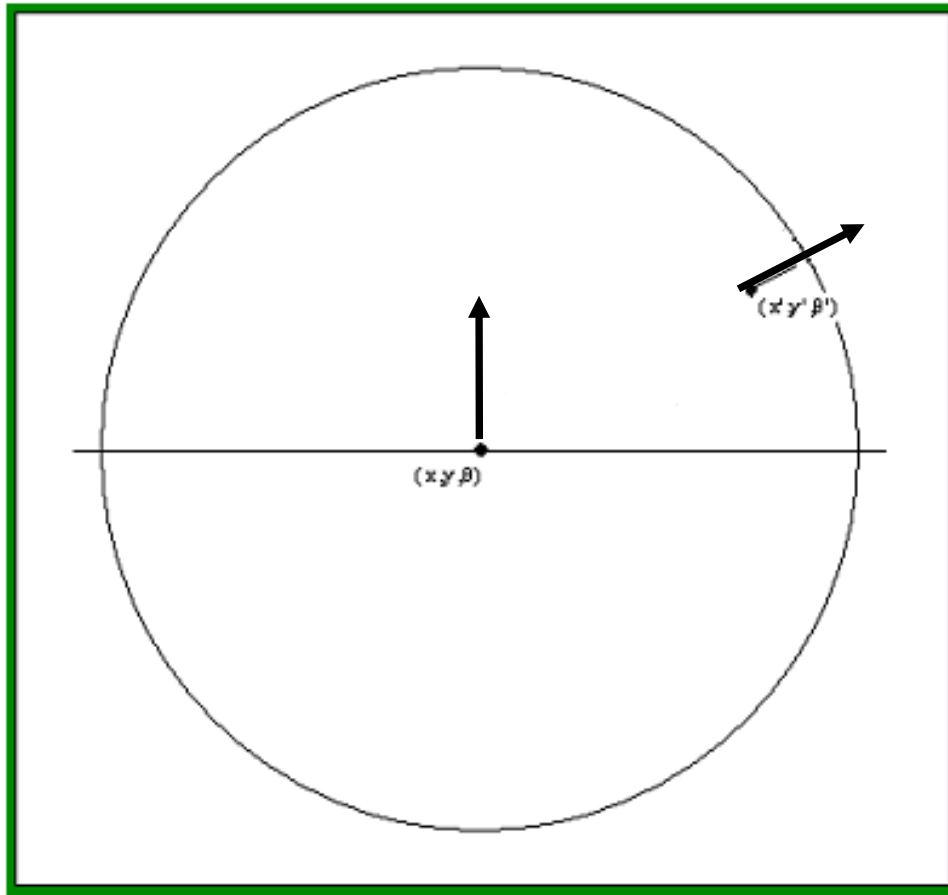
When $\delta\theta \rightarrow 0$, so does d and the cylinder becomes arbitrarily small

Type 2 Maneuver

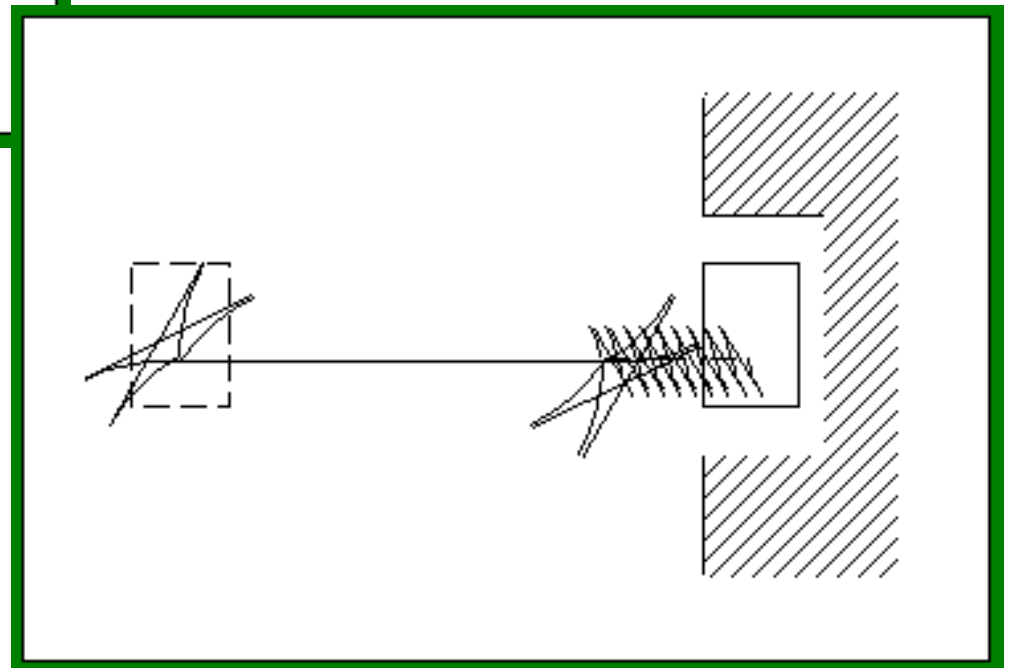
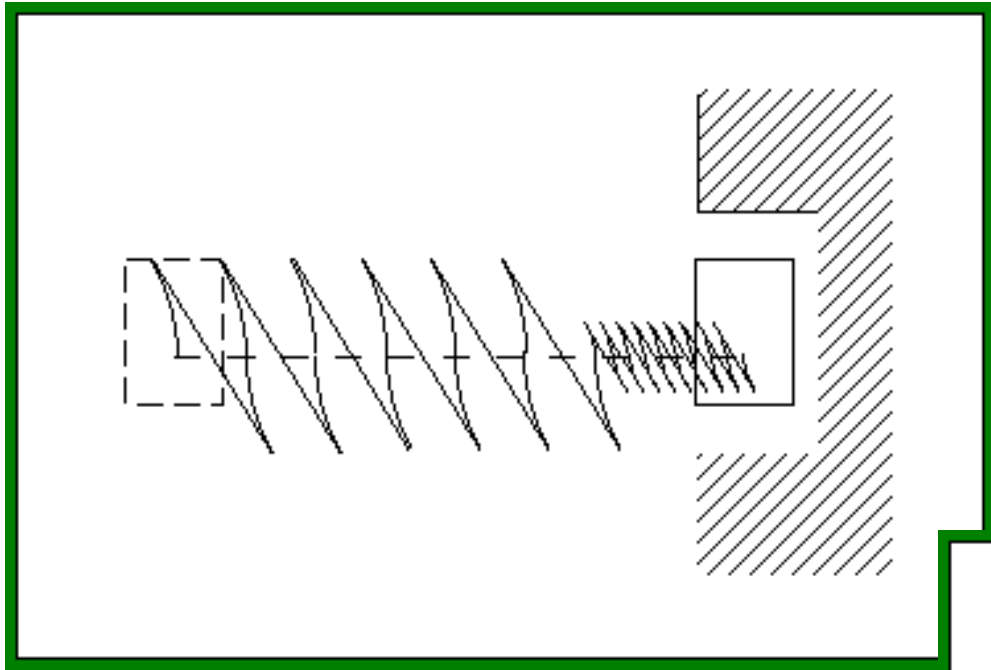


→ Allows pure rotation

Combination

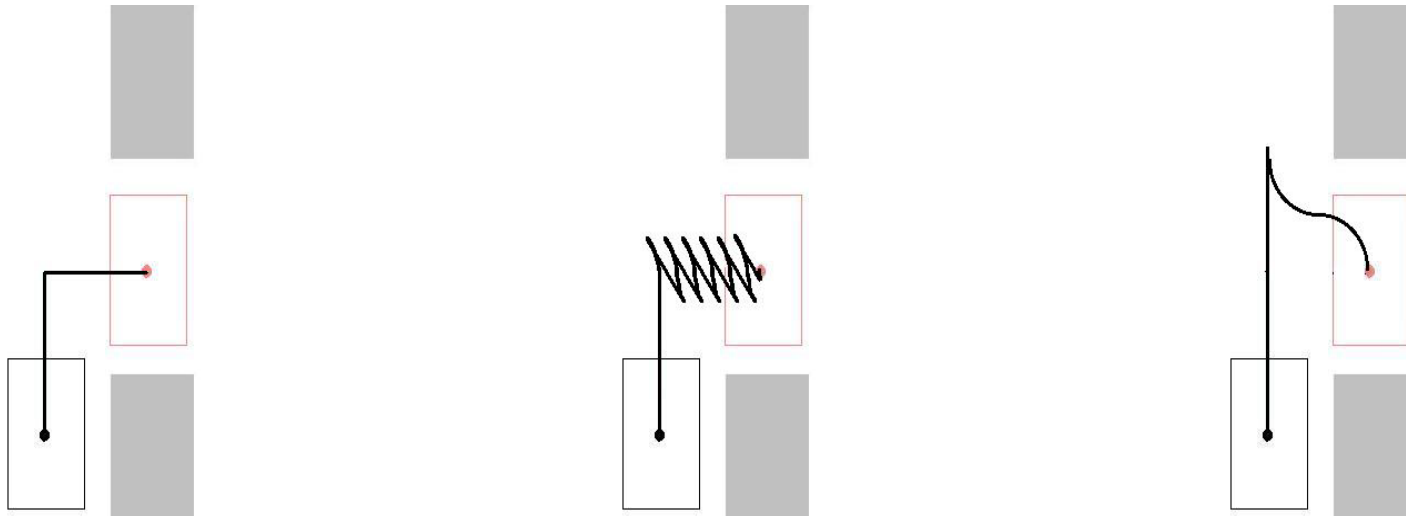


Path Examples



Drawbacks of two-phase Planning

- Final path can be far from optimal
- Not applicable to cars that can only move forward



Reeds and Shepp Paths

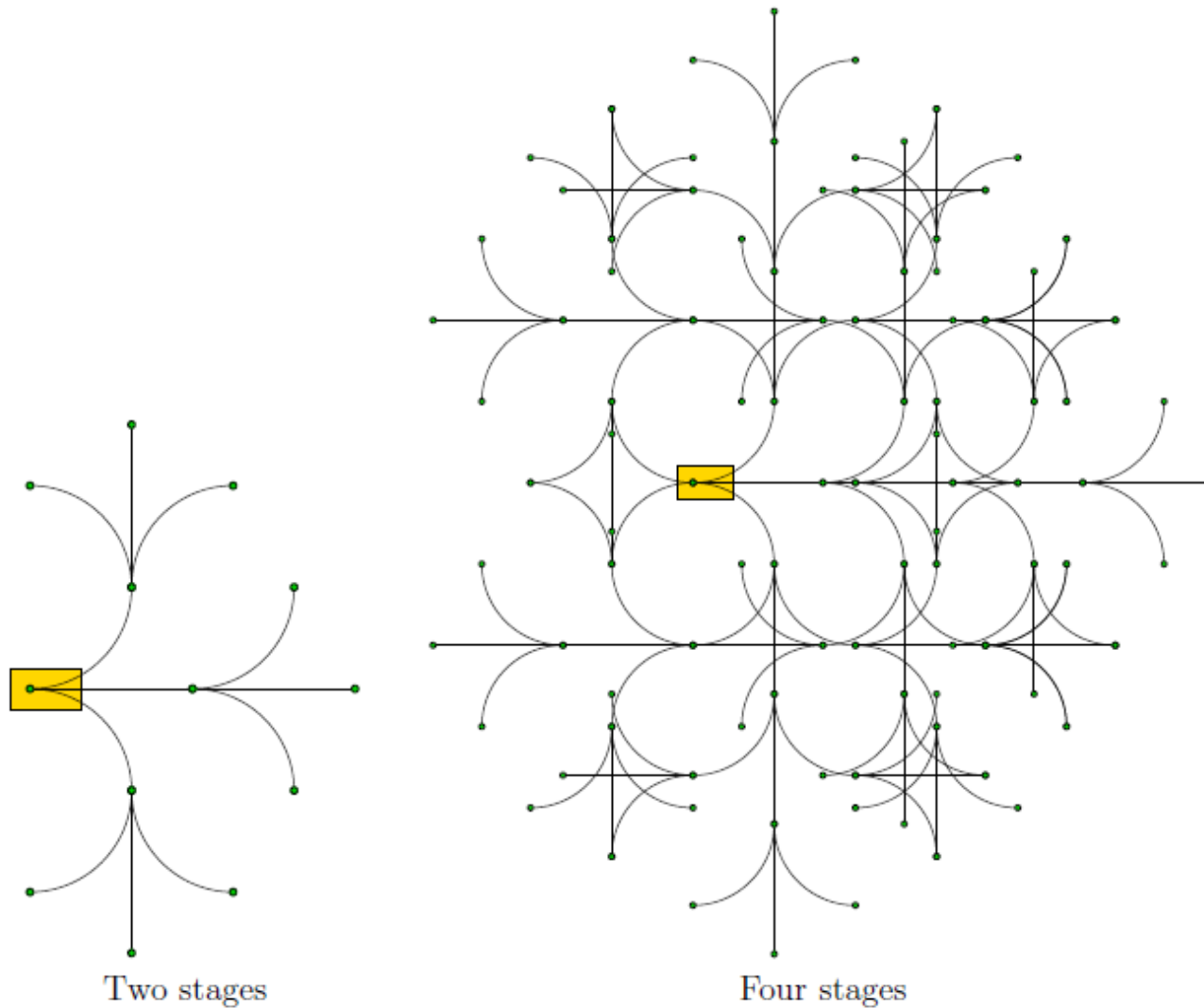
Simple Car: $U = [-1, 1] \times (-\phi_{\max}, \phi_{\max})$. By requiring that $|u_{\max}| \leq \phi_{\max} < \pi/2$, a car with minimum turning radius $\min = L / \tan \phi_{\max}$ is obtained.

Reeds-Shepp Car: Further restrict the speed of the simple car so that $u_s \in \{-1, 0, 1\}$. This model intuitively makes us correspond to three discrete “gears”: reverse, park, or forward.

An interesting question under this model is: What is the shortest possible path (traversed in $\mathbb{R}^2$ by the center of the rear axle) between two configurations in the absence of obstacles?

Dubins Car: Remove the reverse speed $u_s = -1$ from the Reeds-Shepp car to obtain $u_s \in \{0, 1\}$ as the only possible speeds. The shortest paths in $\mathbb{R}^2$ for this car are quite different than for the Reeds-Shepp car

Example – Dubins Car



A reachability tree for the Dubins car with three actions. The k -th stage produces 3^k new vertices.

Control-Based Sampling

- Control-based sampling:
 1. Pick control vector (at random or not)
 2. Integrate equation of motion over short duration (picked at random or not)
 3. If the motion is collision-free, then the endpoint is the new milestone
- Tree-structured roadmaps
- Need for **endgame** regions

Example

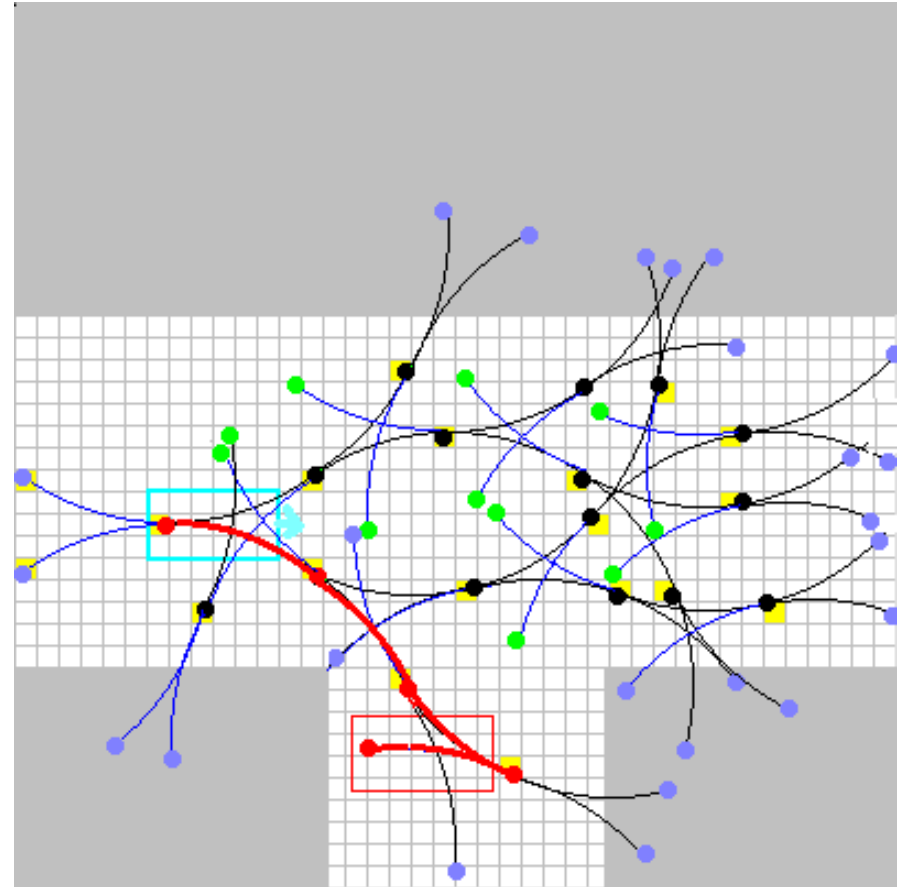
$$dx/dt = v \cos \theta$$

$$dy/dt = v \sin \theta$$

$$d\theta/dt = (v/L) \tan \phi$$

$$|\phi| \leq \Phi$$

1. Select a milestone m
2. Pick v , ϕ , and δt
3. Integrate motion from m
→ new milestone m'



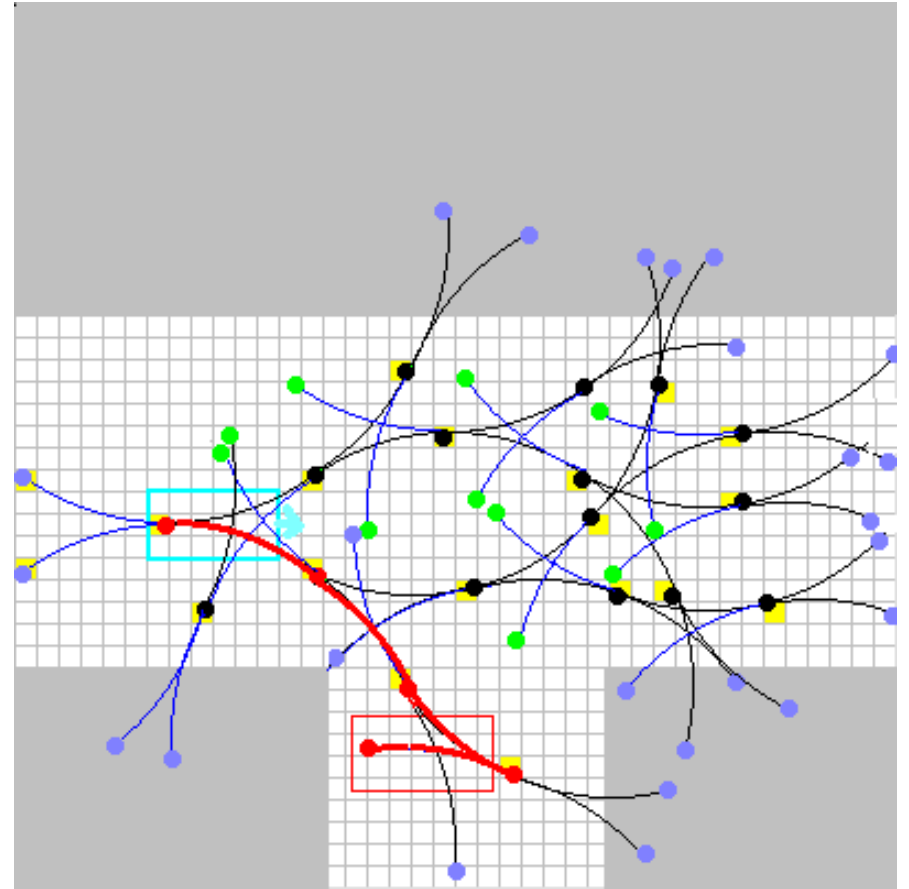
Example

Indexing array:

A 3-D grid is placed over the configuration space. Each milestone falls into one cell of the grid. A maximum number of milestones is allowed in each cell (e.g., 2 or 3).

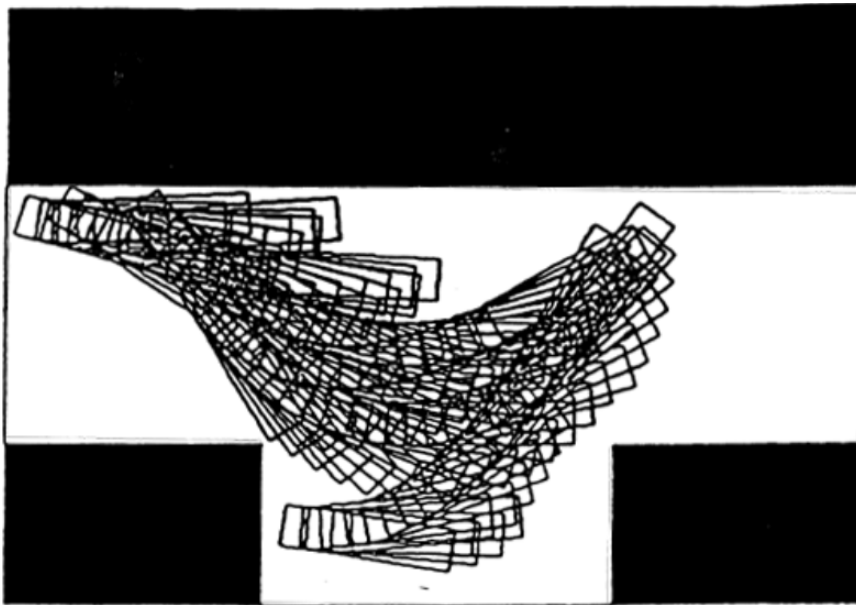
Asymptotic completeness:

If a path exists, the planner is guaranteed to find one if the resolution of the grid is fine enough.



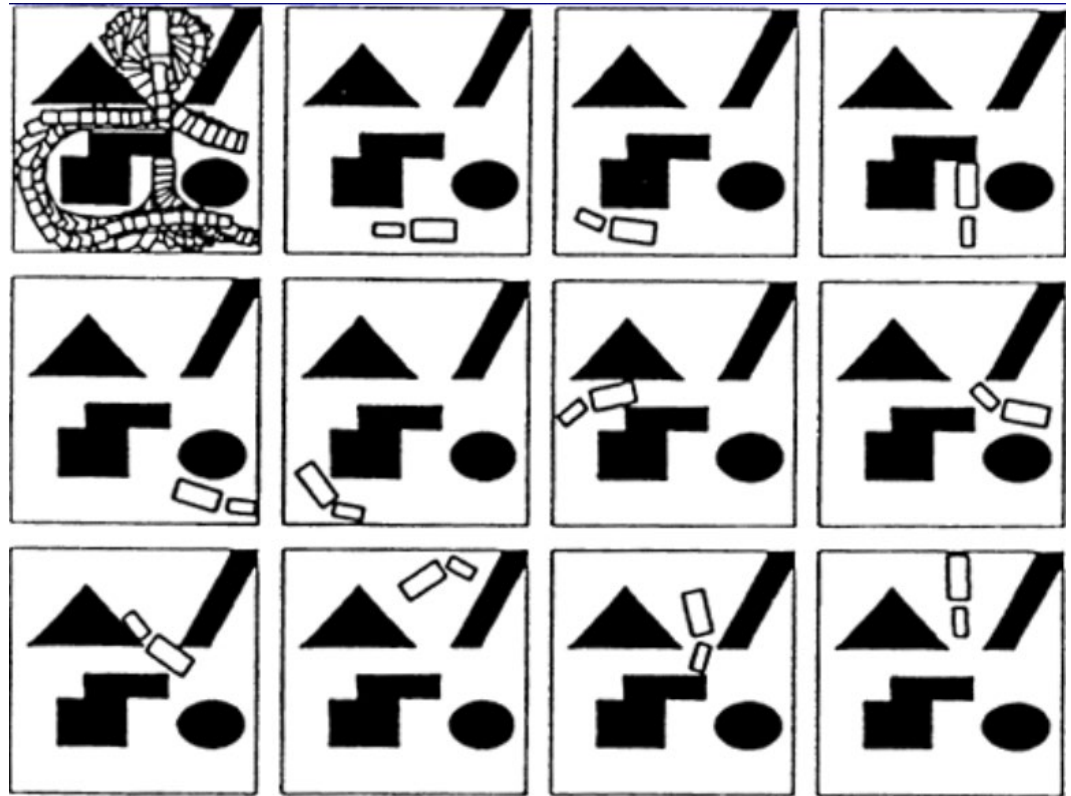
Computed Paths

Car That Can Only Turn Left



$$\varphi_{\max} = 45^\circ, \varphi_{\min} = 22.5^\circ$$

Tractor-trailer



$$\varphi_{\max} = 45^\circ$$

Control-Sampling RRT

Configuration **generator** $f(x, u)$

Build a tree T of configurations

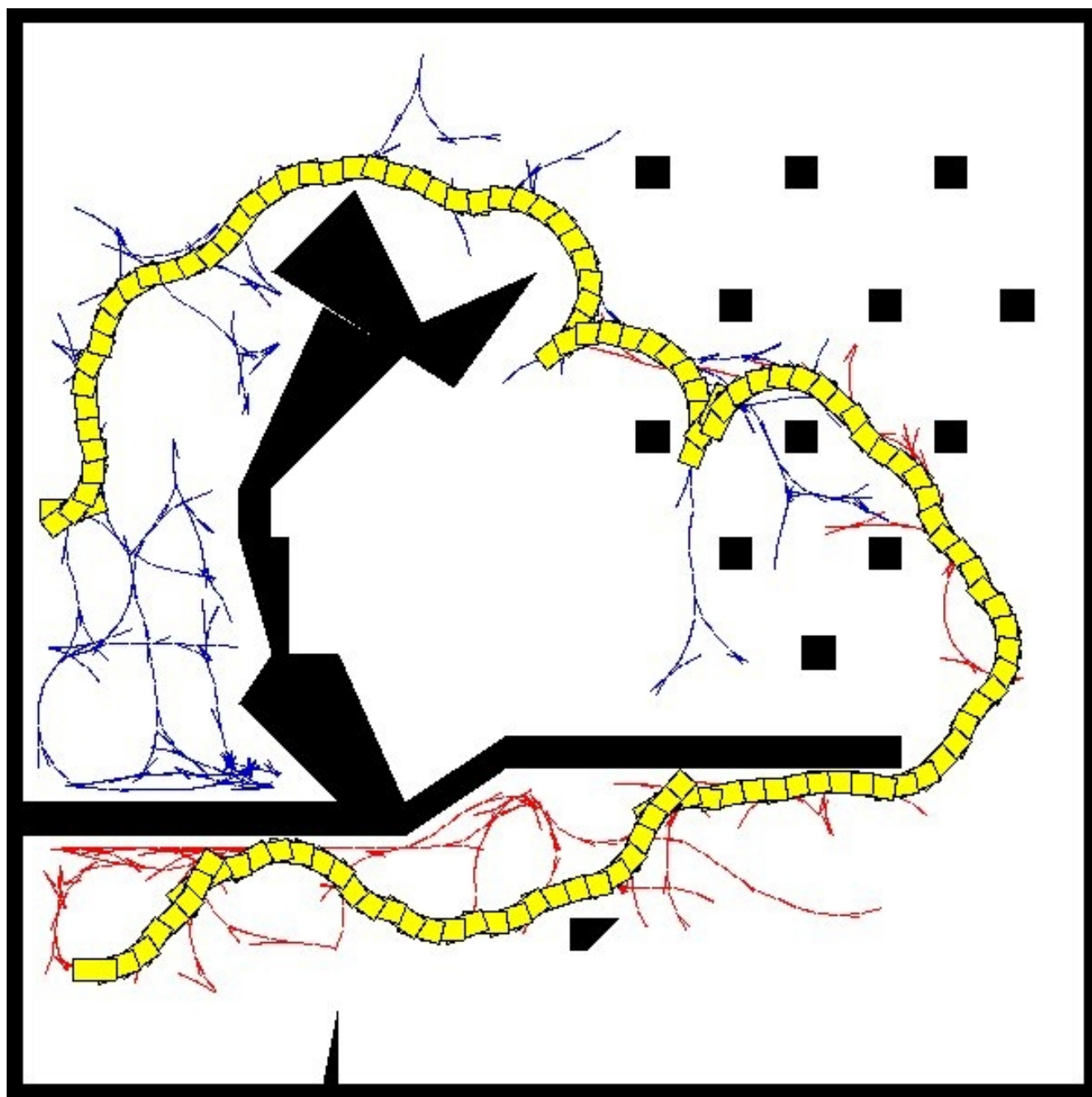
Extend:

- Sample a configuration x_{rand} from X at random

- Find the node x_{near} in T that is closest to x_{rand}

- Pick a control u that brings $f(x_{\text{near}}, u)$ close to x_{rand}

- Add $f(x_{\text{near}}, u)$ as a child of x_{near} in T



Weaknesses of RRT's strategy

Depends on the domain from which x_{rand} is sampled

Depends on the notion of “closest”

A tree that is grown “badly” by accident can greatly slow convergence

